

Distributed Job Scheduler

Intern Assignment Submission

Harshit Ravindra Kulkarni

Roll No: RA2311003011810

1. Source Code & Setup Instructions

GitHub Repository: <https://github.com/kulharshit21/distributed-job-scheduler-submission>

Branch: master

How to Clone and Run (Local End-to-End)

This project is built using a modern Turborepo monorepo structure. The backend utilizes NestJS, the frontend uses React/Vite, and Prisma ORM manages PostgreSQL.

```
# 1. Clone the repository git clone https://github.com/kulharshit21/distributed-job-scheduler-submission.git cd distributed-job-scheduler-submission # 2. Start Infrastructure (PostgreSQL & Redis) docker compose up -d # 3. Install Dependencies npm install # 4. Setup Database & Seed Data npm run db:push -w packages/database npm run db:seed -w packages/database # 5. Run the Application Concurrently (API, Worker, Web) npm run dev
```

- Frontend URL:** <http://localhost:5173>
- API Server:** <http://localhost:3000>
- Demo Credentials:** demo@acmecorp.com / password123

2. Deployment Strategy

The system is architected to be cloud-native and supports zero-downtime distributed deployments.

- Frontend** **Vercel:** The React/Vite SPA is deployed on Vercel for edge-caching, CI/CD integration, and global CDN delivery.
- Backend API & Worker** **Railway:** NestJS API and Worker nodes are deployed independently on Railway to allow independent horizontal scaling.
- Database** **Railway:** Managed PostgreSQL and Managed Redis instances on Railway. (Note: Supabase was explicitly excluded from the tech stack to demonstrate raw database and cache)

handling).

3. System Architecture

The system utilizes a microservices-inspired monolithic repository, featuring decoupled API and Worker services communicating via PostgreSQL (data persistence/locking) and Redis (pub/sub & real-time socket broadcasting).

 Architecture Diagram

Key Architectural Decisions

- **Turborepo Monorepo:** Ensures absolute consistency across Database schema definitions, DTOs, interfaces, and atomic commits across API, Worker, and Web boundaries.
- **Decoupled Worker Subsystem:** The worker service operates completely independently of the API. This isolates intense computational processing from incoming HTTP traffic, preserving API latency SLA.
- **Real-time Telemetry:** Redis Pub/Sub combined with WebSockets provides real-time, low-latency job status updates to the frontend dashboard.

4. Database Design (Entity-Relationship)

The schema is normalized and highly indexed, specifically optimized for distributed concurrent locking and high-throughput query operations.

 ER Diagram

5. Backend Engineering & Concurrency

PostgreSQL over RabbitMQ/Kafka

Instead of deploying a dedicated message broker, the system utilizes PostgreSQL combined with `FOR UPDATE SKIP LOCKED`. This provides massive benefits:

- **Atomic Transitions:** Business logic transactions and queue state changes are executed in a single atomic database transaction.
- **No Distributed Transactions:** Avoids the Two-Phase Commit (2PC) problem inherent when syncing a Database with a Broker.

Concurrency Handling

- **Atomic Locking:** When workers query for jobs, they execute `SELECT ... FOR UPDATE SKIP LOCKED`. This prevents two workers from fetching the same job, completely eliminating race conditions without needing distributed locks like Redlock.
- **Queue-Level Concurrency:** Implemented a `QueueTokenBucket` pattern in the database. Workers dynamically check the active execution count against the `concurrency_limit` defined per queue before claiming.

Reliability Mechanisms

- **Dead Letter Queue (DLQ):** Jobs exceeding `max_retries` are atomically relocated to a DLQ, preserving the payload and failure metadata for developer inspection.
- **Orphan Job Recovery:** A watchdog daemon monitors jobs stuck in the `RUNNING` state where `locked_until` has expired. These are assumed to belong to crashed workers and are automatically requeued.
- **Idempotency:** The API endpoints and worker processors enforce at-least-once delivery, requiring clients to supply idempotency keys.

6. API Design & Documentation

The API is strictly RESTful, versioned, and documented.

Endpoint	Method	Description
/api/v1/projects/:id/queues	POST	Provisions a new queue with defined concurrency limits.
/api/v1/queues/:id/jobs	POST	Enqueues a one-off job. Validates payload via DTOs.

Endpoint	Method	Description
/api/v1/queues/:id/scheduled-jobs	POST	Enqueues a cron job using robust CRON expression parsing.
/api/v1/jobs/:id	GET	Retrieves current job status, execution history, and logs.

Validation: All inputs are sanitized using `class-validator`. Error responses adhere to standard RFC 7807 (Problem Details for HTTP APIs).

7. Frontend & UX (User Experience)

The dashboard is built to provide an absolute top-tier, premium experience that wows the user immediately.

- **Glassmorphism Aesthetic:** Built using React, Vite, and TailwindCSS, the UI incorporates frosted glass effects (`backdrop-blur`) layered over highly curated, vibrant mesh gradients. It feels modern, sleek, and high-tech.
- **Real-time Reactivity:** Utilizing WebSockets (Socket.io), the frontend completely eliminates the need for manual refreshing. Queue throughput metrics, job completions, and system health are streamed live.
- **Micro-animations:** Dynamic hover effects, smooth transitions (via Framer Motion / Tailwind transitions), and tactile button feedback greatly elevate the user interaction.
- **Comprehensive Dashboards:** Includes detailed views for specific Job Logs, Execution Histories, and DLQ management.

(Screenshots of the actual UI are included in the repository README under the visual showcase section.)

8. Testing Strategy

To ensure 99.99% reliability, the system leverages a multi-tiered testing strategy:

- **Unit Testing (Jest):** Granular testing of business logic modules (e.g., Retry backoff calculators, Cron parsers). Dependencies like `PrismaService` are thoroughly mocked.
- **Integration & Concurrency Testing:** Specialized test suites spawn multiple asynchronous workers against a test database to prove that the `SKIP LOCKED` mechanism prevents duplicate job claims under extreme load.
- **End-to-End (E2E) Testing:** Supertest validates the entire HTTP lifecycle, proving that controller validation, JWT authentication guards, and database persistence integrate seamlessly.